古畑ラナと学ぶ
コンピュータ科学

ラナ
プログラムがどう動くか、考えていました。
先生
ラナ、何してるの？
ラナ
……一緒にやってみますか？
先生
私にも分かるかな。
ラナ
はい。紙に書きながら、動きを追うこともできますから。
先生
パソコンも使わずに？
ラナ
まずは、簡単なものからやってみませんか？
先生
うん。どこから見ればいい？

有限オートマトン

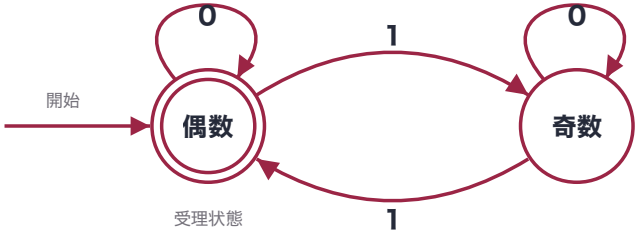
ラナ 先生。0と1からなる文字列に、1が偶数個含まれているかどうか、どうすれば確かめられると思いますか。

先生 一つずつ数えればいいね？

ラナ はい。ただ……偶数か奇数かがわかればいいので、正確な個数はわからなくても構いません。

ラナ 1を一つ読むたびに、偶数、奇数、偶数……と変わりますから。

01 **1を読んだら、反対側へ**



矢印の数字は読む文字、二重丸は受理状態です。

ラナ それぞれを一つの状態で表すと、こうなります。1を読むたびに、反対側へ移るんです。

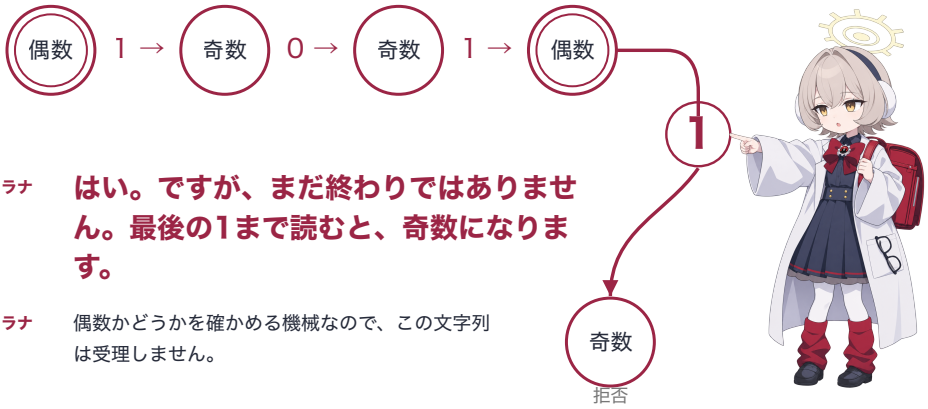
先生 0を読んだら？

ラナ そのままです。1の個数は増えていませんので。

ラナ 最初はここ、偶数から始めます。まだ1を読んでいないので、0個です。

ラナ 1011を入れてみましょう。最初は偶数。1を読むと奇数、次は0なので、そのまま……。

先生 その次が1だから、また偶数だね。



ラナ **はい。ですが、まだ終わりではありません。最後の1まで読むと、奇数になります。**

ラナ 偶数かどうかを確かめる機械なので、この文字列は受理しません。

先生 途中で偶数になったからって、止めちゃいけないんだね。

ラナ はい。その後に、まだ1があるかもしれませんから。

ラナ **このように、決まった数の状態の間を移りながら入力を読む機械を、有限オートマトンといいます。**

先生 もっと長い文字列でも、状態は二つでいいの？

ラナ はい。読む時間は長くなりますが、状態を増やす必要はありません。

$$0*(10*10*)*$$

ラナ この機械が受理する文字列を、正規表現で書くとこうなります。

ラナ 括弧の中に1が二つありますね。このまとまりを繰り返すと、1は必ず偶数個になります。間に0がいくつ入っても構いません。

空文字列 ε

移動せずに受理

先生 何も入力しなかったら？

ラナ それも受理します。最初の偶数の状態から、動いていませんので。

ラナ ……入力を忘れたのか、わざと空にしたのかは、この機械にはわかりませんが。

※ ここでの正規表現は形式言語理論での意味です。後方参照などの拡張は含みません。

括弧を閉じる順番

受理

拒否

ラナ　先生。今度は括弧です。この二つは開き括弧と閉じ括弧の数が同じですが、どちらも正しいでしょうか。

先生　二つ目は、括弧の順番がおかしいね。

ラナ　はい。数が合うだけではいけません。括弧を開いたり閉じたりする順番が正しいかも、確かめる必要があります。

まだ閉じていない括弧の数

先生　その数を状態にすればいいんじゃない？ 0個、1個、2個……って。

ラナ　はい。括弧の入れ子が二重までなら、そう作れます。

先生　三重なら、状態をもう一つ増やして？

ラナ　はい。ですが、何重まで入るか決めないと、どこまで状態を用意すればいいでしょうか。

先生　どれだけ増やしても、足りなくなるかもしれないね。

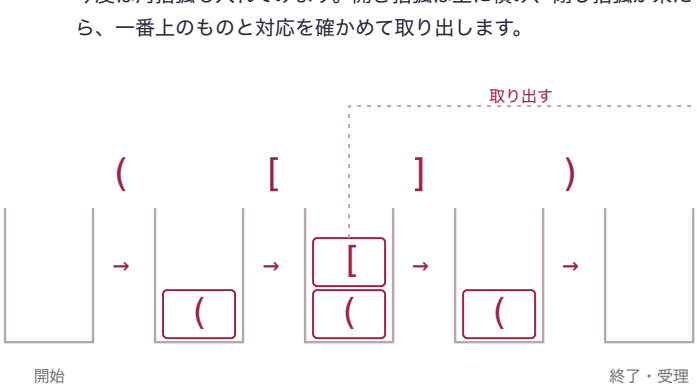
ラナ　**そのとおりです。深さに制限のない括弧の対応は、有限オートマトンだけではすべて確かめられません。**

ラナ　今度は、まだ閉じていない括弧を別に積んでおきます。

先生　状態のほかに、覚えておく場所を用意するんだね。

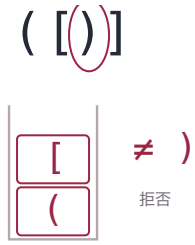
ラナ　はい。これなら、括弧が増えると積んだほうが高くなります。

ラナ　今度は角括弧も入れてみます。開き括弧は上に積み、閉じ括弧が来たら、一番上のものと対応を確かめて取り出します。



先生　後から開いた角括弧から閉じるんだね。

ラナ　**はい。このように、一番上に入れて一番上から取り出す記憶構造を、スタックといいます。**



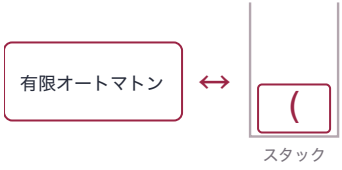
先生　じゃあ、こっちは？ 種類ごとの数は合っているけど。

ラナ　今閉じようとしているのは丸括弧ですが、一番上には角括弧が残っています。先にそちらを閉じる必要があります。

ラナ　取り出す括弧がなかったり、種類が違ったりしたら拒否します。最後まで読んでも残っていたら、それも対応が取れていません。

) 取り出す括弧がない

(( ) 最後に (が残る



先生　途中で間違いがなくて、最後に空になればいいんだね。

ラナ　はい。有限オートマトンにこのようなスタックを付けた機械を、プッシュダウンオートマトンといいます。

先生　括弧一つ抜けただけでも、見逃してくれないんだね。

ラナ　はい。先生がどこまでをまとめるおつもりだったのか、私が勝手に決めるわけにはいきませんので。

※ 理論上、スタックの大きさに制限はありません。丸括弧一種類だけなら、負にならないかを調べるカウンタでも十分です。

文字を読んで計算する

2 + 3 * 4

* は掛け算

ラナ 先生。今度はこの式をコンピュータに計算させてみましょう。何から始めればいいでしょうか。

先生 掛け算からだね。答えは14。

ラナ はい。ですが、コンピュータにはまず文字が入ってきます。どれが数字で、どれが計算の記号なのか、そこから区別する必要があります。

数値 2	+	数値 3	*	数値 4

12
↓
数値 12

ラナ このように、読み取る単位に分けたものをトークンといいます。数値や記号を見分けるこの処理が、字句解析です。

先生 ただ一文字ずつ区切るわけじゃないんだね。

ラナ はい。12なら二文字をまとめて、一つの数値として読みます。ここでは空白も読み飛ばします。

2 + 3 * 4

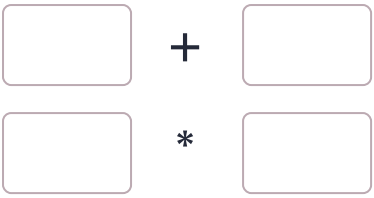
先にまとめる

先生 これで左から計算すればいい？

ラナ それでは2と3を先に足して、答えが20になります。先ほど先生は、掛け算からなさいましたね。

先生 コンピュータにも、その順番を教えないといけないね。

ラナ はい。掛け算を足し算より先にするという、決められた規則に従わせる必要があります。このように、規則に従ってトークンの構造を読み取る処理を、構文解析といいます。

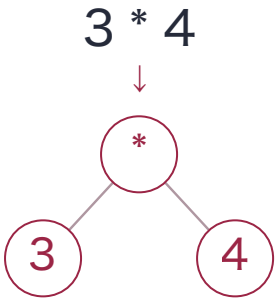


左と右
対象を一つずつ

ラナ　ここでの足し算と掛け算には、計算する対象が二つずつ必要です。記号の左と右に、一つずつです。

先生　数字が入る場所だね。

ラナ　はい。式が入ることもあります。まず掛け算を見ると、左には3、右には4があります。

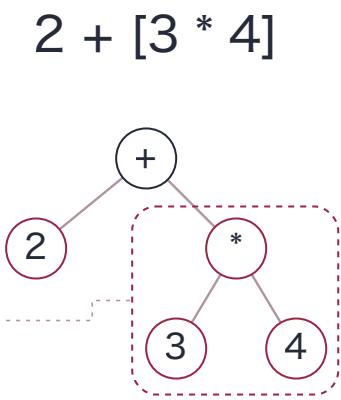


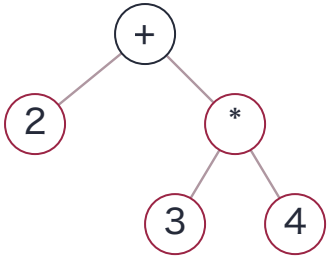
先生　掛け算の記号につながる二つの数を、下に描いたんだね。

ラナ　はい。次に足し算を見ます。左には2があり、右には先ほどまとめた掛け算全体が入ります。

先生　右の場所に、数一つの代わりに小さな図が丸ごと入るんだね。

ラナ　はい。このように式の構造を表したものを、構文木といいます。まだ計算はしていません。何と何を計算するか、つないでおいたんです。





構文木

先生　足し算が一番上にあるのに、掛け算からなの？

ラナ　はい。上の足し算をするには、右側の値が必要です。ですから、その下の3掛ける4を先に計算します。

3 * 4 → 12

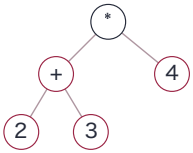
2 + 12 → 14

値に置き換える

先生　そこに12が入って、最後に2を足すと14だね。

ラナ　はい。式の値を求めることを、評価といいます。今は木の構造に沿って直接計算しました。

(2 + 3) * 4



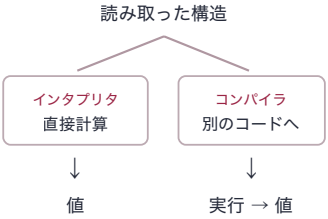
→ 20

先生　括弧で2足す3をまとめたら？

ラナ　今度は足し算のほうを先にまとめます。木も変わり、結果は20になります。

ラナ　このように、読み取った構造に従って計算を実行するプログラムを、インタプリタといいます。

ラナ　すぐに計算せず、別の形のコードへ変換することもできます。その役目を担うプログラムが、コンパイラです。



先生　計算する内容を、別のコードで書いてくれるんだね。

ラナ　はい。変換したコードを実行すると、計算が行われます。今回は式を読み、直接計算するほうを作ったんです。

2 + 3 * 4 → 20

足し算優先の規則

先生　でも、掛け算からって決めたのも私たちだよね。足し算から計算する電卓も作れるの？

ラナ　はい。足し算を先にまとめる規則に変えれば作れます。

先生　じゃあ、2足す3掛ける4が、括弧なしでも20になるんだね？

ラナ　はい。……ただし、これは先生専用と表示しておきます。他の方がお使いになったら、かなり戸惑われそうですね。

※ ここでは数値・足し算・掛け算・括弧だけの小さな言語を扱います。インタプリタが必ず木を直接実行するとは限らず、コンパイルと解釈を組み合わせる実装もあります。

スタックでは足りないなら

abc
aabbcc
aaabbbccc

受理

aabbc

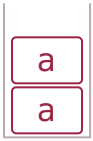
拒否

ラナ 先生。今度はa、b、cがこの順に並び、数もすべて同じか確かめてみましょう。

先生 aとbはスタックで合わせられるね。aを積んで、bが来たら一つずつ取り出して。

ラナ はい。ですが、aとbをすべて合わせると、スタックは空になります。

aを積み、bごとに取り出す方法



aaを読んだ後

bb →

bbを読んだ後

cc

比べる個数は？

先生 cも二つ必要なのに、何個だったか残っていないね。

ラナ そのとおりです。この方法では、cと比べる個数を残せません。

先生 積み方を変えたらどうかな？

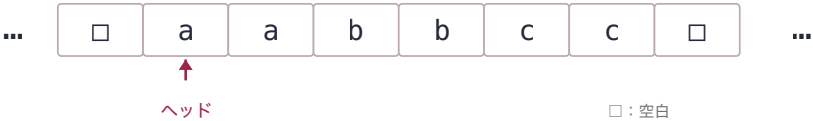
ラナ この方法だけの失敗なら、別の方法を探せますね。ですが、この問題はスタックを一つ使うプッシュダウンオートマトンでは解けないことが知られています。

ラナ 個数に上限を設けず、すべての入力を確かめる場合の話です。証明は……ここでするには、ページが足りませんね。

ラナ 今度は、文字を読んで通り過ぎた後にも、戻って印を付けられるようにしてみましょう。

□	a	a	b	b	c	c	□

※ 入力を一方向に読み、スタックを一つ使う標準的なプッシュダウンオートマトンを指します。不可能性の証明は省略し、失敗例は直感的な説明です。



状態：aを探している

先生　今度はマスが横に並んでいるね。

ラナ　テープと呼びます。一マスに記号を一つ書き、ヘッドが指しているマスを読みます。

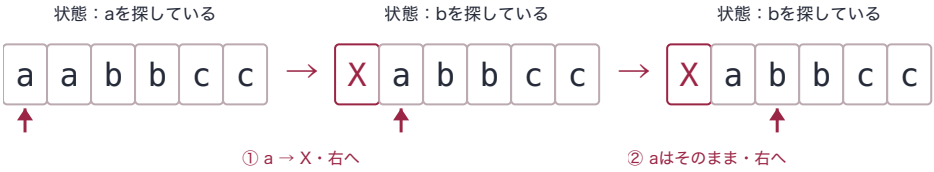
先生　読んだ文字を書き換えてもいいの？

ラナ　はい。そのマスに記号を書き、左か右へ一マス動けます。

ラナ　動作の規則も決めておく必要があります。今の状態と読んだ記号に応じて、何を書き、どちらへ動き、次にどの状態になるかを、この状態遷移表にまとめました。

状態遷移表・最初のaとbに印を付ける規則の一部

	現在の状態	読む記号	書く記号	移動	次の状態
①	aを探している	a	X	右	bを探している
②	bを探している	a	a	右	bを探している
③	bを探している	b	Y	右	cを探している



同じaを読んでも、状態によって動作が変わります。

先生　状態はさっきと同じようにあって、覚えておく場所がテープになったんだね。

ラナ　**はい。テープとヘッド、そして状態に応じた動作規則を備えた計算のモデルを、チューリングマシンといいます。**

先生　テープの端に着いたら？

ラナ　理論上はテープの長さに制限がないので、必要なだけ空白のマスを使えます。ただし、遠くのマスへ行くには、その分だけ動く必要があります。

ラナ　aabbccで試しましょう。まず、a、b、cが順番にまとまって並んでいるか確かめます。

ラナ　次に、まだ印を付けていないa、b、cを一つずつ探し、それぞれX、Y、Zに書き換えます。

文字を探して印を付ける区間の要約

先生　一つずつ対応を取った印だね。

先生　左へ戻るのも、規則に書いてあるの？

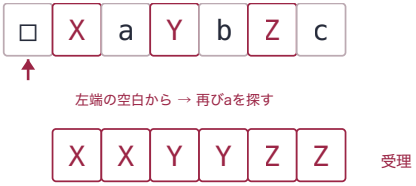
ラナ　はい。cをZに変えたら、状態を「左へ戻る」に変えます。左端の空白に着くまで、一マスずつ戻るのです。

ラナ　空白に着いたら、またaを探します。Xは確認済みなので、そのまま通り過ぎます。

左へ戻って再開する規則・一部

現在の状態	読む → 書く	移動	次の状態
cを探している	c → Z	左	左へ戻る
左へ戻る	s → s	左	左へ戻る
左へ戻る	□ → □	右	aを探している
aを探している	X → X	右	aを探している

sはa、b、c、X、Y、Zのいずれか。s → sは読んだ記号をそのまま残します。



先生　二回目が終わると、全部に印が付くね。

ラナ　はい。a、b、cが残っていないので受理します。途中で対応する文字が足りなかったり、最後にbやcが残ったりしたら拒否します。

先生　文字を消してなくす代わりに、確かめた場所に印を残したんだね。

先生　私が手でなぞっても、同じ答えになるんだね。

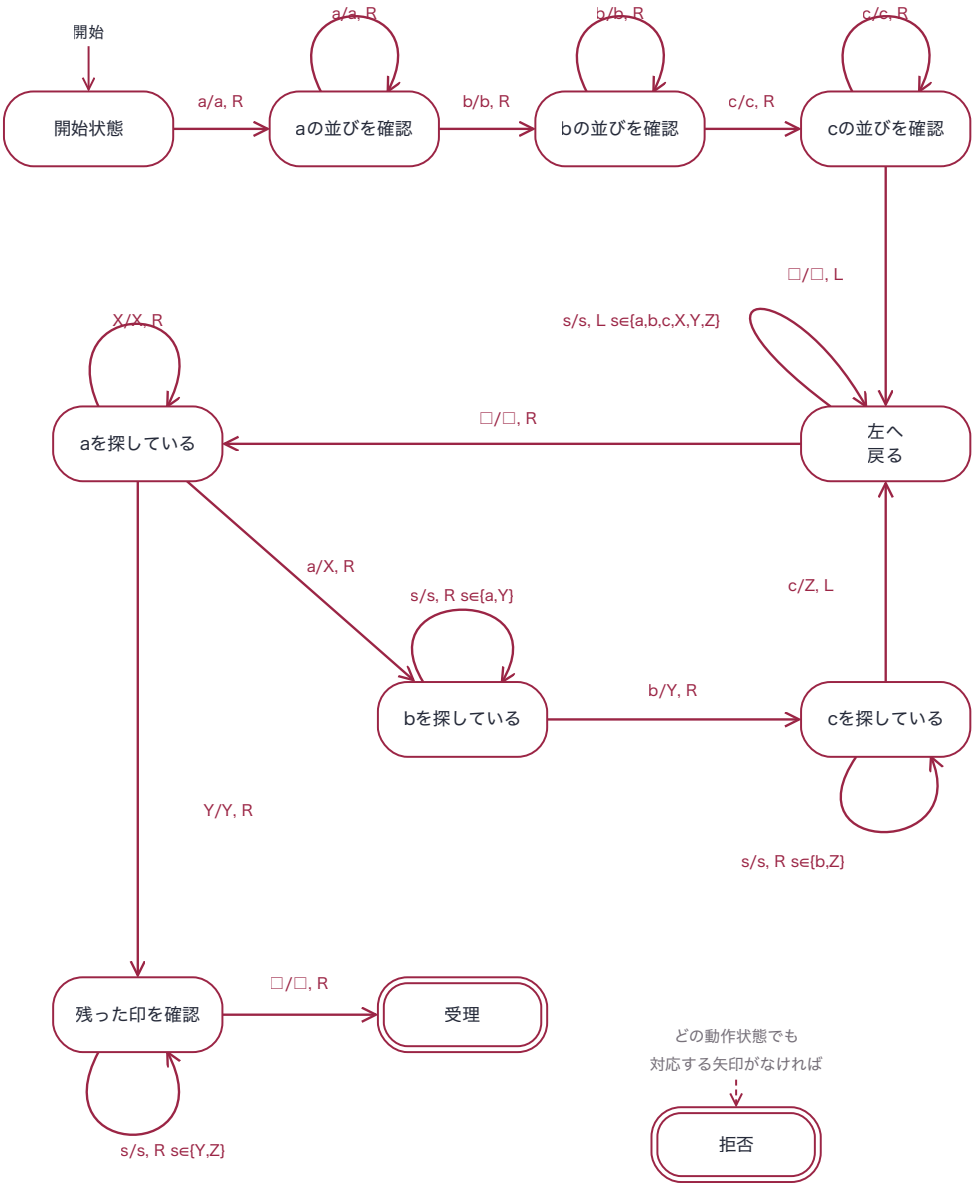
ラナ　はい。紙と鉛筆をご用意いただけますか。私が規則を読み上げます。

先生　説明を聞いていたはずが、いつの間にか私がコンピュータになった気分だよ。

※ テープの変化は区間ごとの要約です。実際のマシンは一度に一マスを読み書きして移動します。

この例の状態遷移図

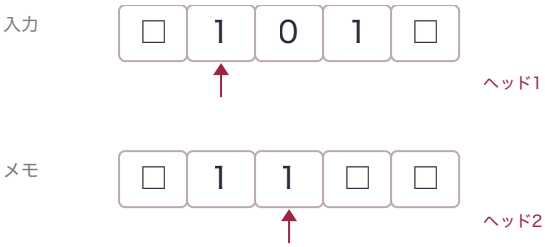
矢印：読む記号 / 書く記号, 移動方向・L = 左、R = 右
s/sは記号をそのまま残します。s∈{…}は括弧内の各記号の規則をまとめた表記です。
開始時のヘッドは入力の先頭。入力の両側はすべて空白□です。



矢印がない場合：記号をそのまま残し、右へ一マス動いて拒否します。
受理・拒否状態では停止します。空文字列も開始状態で拒否します。

※ 入力はa、b、cのみ。ここでは各文字が一つ以上の場合（n ≥ 1）を扱います。

テープを増やしたら？



ラナ　先生。今度はテープをもう一本置いてみます。片方で入力を読んで、もう片方をメモに使うのはどうでしょうか。

先生　紙の上を行ったり来たりするのが、少し減りそうだね。

ラナ　はい。テープごとにヘッドを置いて、別々に動かせます。こうした機械を、多テープチューリングマシンといいます。

先生　それなら、何が変わるの？

ラナ　メモ用の場所を別に使えるので、計算の手順を組み立てやすくなります。ただし、テープが一本のときにはできなかった計算まで、できるようになるわけではありません。

二本の内容を一列に記録

ラナ　二本のテープの内容を、こうして一本に区切って書くこともできます。それぞれのヘッドがどこにあったかも、印を付けておきます。

先生　その代わり、印の付いた場所を探して回るんだね。

ラナ　はい。片方の機械が一度動く間に、もう片方は何度も動くことがあります。それでも、同じ計算をたどることはできます。

先生　できることが増えるわけではないんだ。

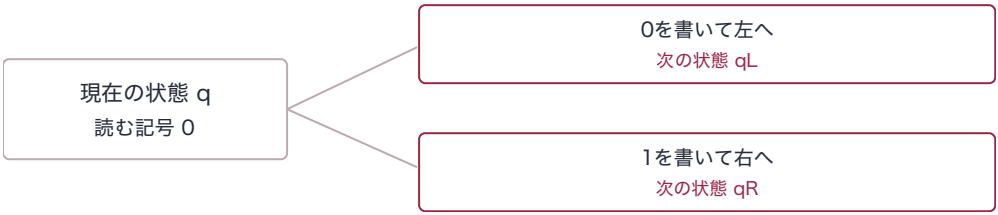
ラナ　そのとおりです。逆に、テープが何本もあるなら、一本だけ使えばよいですね。互いの計算を模倣できるという意味で、この二つのモデルはチューリング同値です。

※ ここでの同値は、計算できる範囲についての言葉です。同じ計算をしても、必要な計算ステップ数や使用する記憶領域の大きさは異なる場合があります。

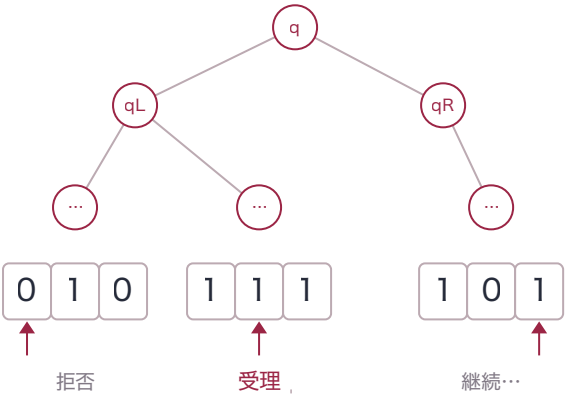
ラナ　今度は規則を変えてみます。同じ状態で同じ記号を読んでも、次の動作がいくつもあったらどうでしょうか。

先生　どちらに進めばいいの？

ラナ　選べる動作ごとに、計算が枝分かれすると考えてください。こうしたモデルを、非決定性チューリングマシンといいます。



枝分かれした計算・概念図



先生　片方では正しいと言って、もう片方では違うと言ったら？

ラナ　受理する枝が一つでもあれば、その入力を受け入れます。すべての枝で一致する必要はありません。

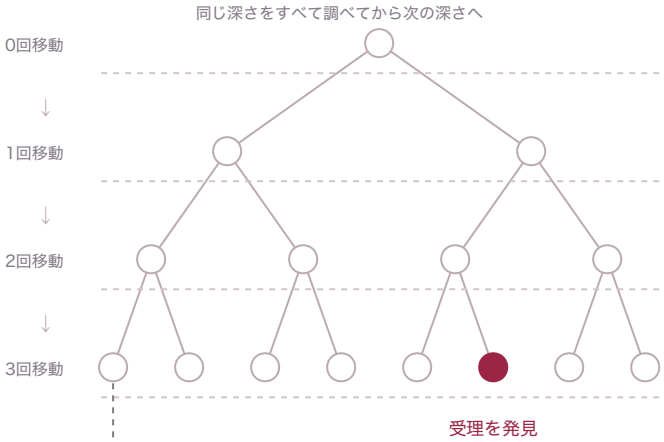
先生　コインを投げて片方だけ進んだら、正しいほうを見逃すかもしれないね。

ラナ　はい。ここでは、選ばれる確率を考えているのではありません。受理する計算が一つでもあるかを見ています。

※ 受理する枝がなければ、その入力は受け入れません。ただし、必ず拒否して止まるとは限りません。いつまでも続く枝がある場合もあります。

先生　さっきの機械でも、これをまねできるかな。分かれ道では一つずつ選ぶんでしょう？

ラナ　枝分かれした計算を、テープに記録しておきます。一回動いて行けるところをすべて見て、次に二回、三回動いて行けるところを見るのです。



先生　一つの道を最後まで進んでから戻るわけではないんだね。

ラナ　はい。その道がいつまでも続いていたら、ほかの道を調べられませんから。こうして順に調べれば、受理する枝があるときには、いつか見つけられます。

先生　一つずつたどっても、受け入れる入力は同じになるんだ。

ラナ　はい。非決定性チューリングマシンも、先ほどの決定性チューリングマシンと同じ言語を認識できます。ただし、たどる計算がとても多くなることがあります。

先生　一つずつやると、書いておくことがどんどん増えるね。

ラナ　はい。先生がお一人でも、たどることはできます。……紙を追加で持ってきます。

先生　休憩の時間も入れてほしいな。

※ 非決定性は、実際の装置が無限個の計算を同時に終えるという意味ではありません。上の説明では、可能な計算を順に探索しています。

どこまで短く書けるでしょうか？

00000000000000000000000000
0000000000 … 0000000000

0が10,000個

ラナ　先生。このファイルと同じものを作るには、0を一万個、全部書く必要があるでしょうか。

先生　コピーすればいいんじゃないかな？

ラナ　元のファイルを使わずに作るとしたら？

先生　それなら……0を一万回書くように命令すればいいね。

ラナ　はい。結果を全部書くより、作り方を書くほうが短く済みますね。

0を10,000回出力
そして終了

→

000…000

プログラムの動作を説明したカード　　同じ結果

先生　同じファイルを作る方法は、いろいろありそうだね。

ラナ　はい。それぞれの方法を、プログラムとして書けます。その中で最も短いプログラムの長さを、コルモゴロフ複雑性といいます。

先生　見た目がどれだけ複雑かではなくて、作り方がどれだけ短いかなんだね。

ラナ　はい。ここでは、先ほどのチューリングマシンを思い浮かべてください。今度はテープに書かれたプログラムを読んで実行するように、規則を決めておきます。プログラムは0と1で書き、何文字あるかを数えます。

※ プログラムの長さは、すべて同じ表記の規則に従って0と1で書いて比べます。特定の文字列だけを短く作るために、新しい命令を追加したり、実行するチューリングマシンの規則を変えたりはしません。必要なデータも文字数に含め、プログラムは別の入力なしで、その文字列だけを出力して停止するものとします。実行時間は数えません。

先生　うまく探せば、どんなファイルでも短く作れるんじゃないかな？

ラナ　できるかどうか、数えてみましょうか。0と1で作れる3文字の文字列は、この8個です。

作る文字列・3文字

000	001	010	011
100	101	110	111

より短く書けるもの・7個

空文字列	0	1	00
01	10	11	

先生　それより短く書くなら、二文字までだね。

ラナ　はい。何も書かない場合まで数えても、7個しかありません。

先生　8個に一つずつ割り当てるには、一つ足りないね。

ラナ　そのとおりです。一つのプログラムが出力して終わる結果は一つです。すべての文字列に、それより短いプログラムを一つずつ割り当てることはできません。

先生　7個とも、使えるプログラムとは限らないだろうしね。

ラナ　はい。作れる結果は、さらに少ないかもしれません。四文字、五文字に増やしても同じです。それより短く書けるものを全部集めても、作りたい文字列に一つずつ割り当てるには、いつも一つ足りません。

先生　どれだけ探しても、短く作れない文字列が残るんだね。

ラナ　はい。そうした文字列を、圧縮不可能な文字列といいます。

※ ここでの「圧縮不可能」は、決めておいた機械では、元の文字列より短いプログラムで作れないという意味です。見た目が不規則だというだけでは、圧縮不可能とは判断できません。

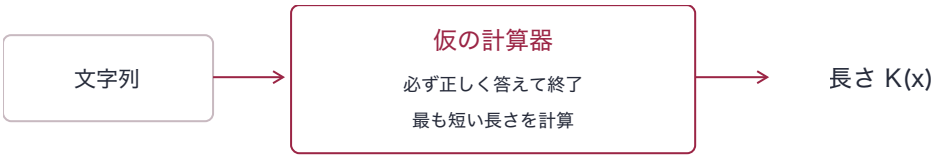
先生　それなら、ファイルを渡せば、それを作るプログラムを何文字まで短くできるかも分かるの？

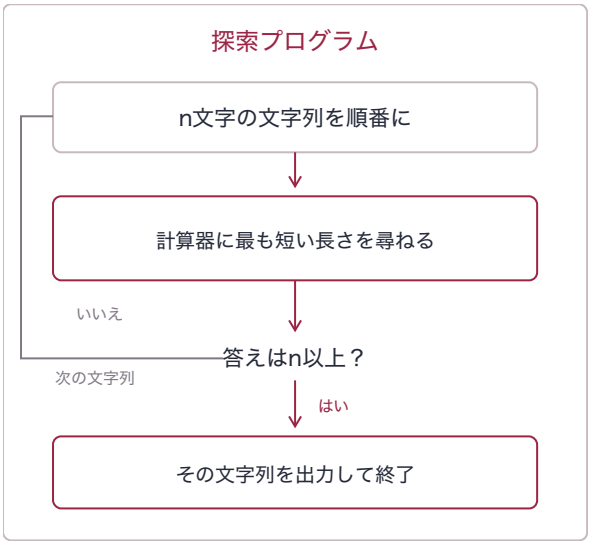
ラナ　どんなファイルでも正確に教えてくれるプログラムは、作れません。

先生　作り方が多すぎるから？

ラナ　それだけが理由ではありません。計算器が一つあると考えてみましょうか。

ラナ　時間がかかっても構いません。どんな文字列を入れても、最も短い長さを教えて終わるプログラムがあると仮定してみましょう。





ラナ　0と1でできたn文字の文字列を、順に入れてみます。答えがn未満なら次へ進み、n以上なら、その文字列を出力して終わります。

先生　短く作れないものを探すんだね。でも、必ず見つかるの？

ラナ　はい。先ほど、そうした文字列は必ず残ると分かりましたね。計算器も毎回答えて終わると仮定したので、いつか見つかります。

先生　計算器さえあれば、探し方自体は難しくないね。

ラナ　はい。では、この探索プログラムの長さを数えてみます。

※ K(x)は、先ほど決めた機械に対するコルモゴロフ複雑性です。計算器を実際に作ったのではなく、存在すると仮定しています。

先生　中に入れた計算器が、とても長いかもしれないよ。

ラナ　はい。そのプログラムの文字数も、全部数えます。それでも、一度作れば、探す文字列の長さnを変えるたびに、計算器を作り直す必要はありません。

先生　書き換えるのはnだけだね。

計算器＋順に探す方法
この部分の長さは同じ

＋

nを書く部分
8 → 1000

両方を数えたものが、プログラム全体の長さ

ラナ　はい。nも0と1で書いておきます。例えば8は1000、16は10000です。値は倍なのに、書く文字は一つ増えるだけですね。数の終わりを区切る印まで含めても、nを十分大きくすれば、プログラム全体をn文字より短くできます。

見つけた文字列の条件
最短でもn文字以上
これより短く作れない

矛盾

今作った探索プログラム
全体でn文字未満
同じ文字列を出力して終了

nが十分大きいとき、この二つは両立しません。

先生　待って。そのプログラムが見つけたのは、n文字より短く作れない文字列だったよね。

ラナ　はい。ところが今、その文字列を作る、もっと短いプログラムができてしまいました。計算器が必ず正しく答えるという仮定は、成り立たないのです。

先生　短い方法を一つ見つけることと、それが一番短いと分かることは、違うんだね。

ラナ　はい。個々の文字列では最小値が分かる場合もありますが、すべての文字列について最小値を計算するプログラムはありません。

先生　それなら、さっきのカードにも「最短」とは書けないね。

ラナ　「短い方法」と書いてください。……「最も」の一言を付けるのが、こんなに難しいのです。

※ 計算器と探索の手順を書いた部分の長さは変わりません。探す文字列の長さを倍々に増やしても、その長さを0と1で記録する部分は一文字ずつしか増えません。数を区切る印まで含めても、十分長い文字列を探すようにすれば、探索プログラム全体が、その文字列より短くなります。

なぜ計算できないのでしょうか？

文字列xを出力して停止する最短のプログラムの文字数を、K(x)と書きます。プログラムは先ほど決めた機械で実行し、同じ規則で0と1で書いて比べます。

① 最短の長さを教える計算器を仮定します。

どんな文字列xでも、正確なK(x)を答えて終了する計算器があると仮定します。速く答える必要はありません。この計算器で長さnの文字列を順に調べ、K(x)がn以上となる最初の文字列を出力して終了します。

② 探す文字列は、必ずあります。

0と1でできた文字列の個数を比べると、次のようになります。一つの短いプログラムが作る結果は一つだけなので、すべてのn文字の文字列を、より短いプログラムで作ることはできません。

長さnの文字列 2^n ＞ それより短い文字列 $1 + 2 + \cdots + 2^{n-1} = 2^n - 1$

作る結果のほうが多いので、K(x) ≥ nとなる文字列が残ります。

③ 探す方法も、一つのプログラムです。

計算器、探索の手順、数を読み取る規則まで含めた固定部分の長さをcとします。ここにnを書き込めば、別の入力なしでxを探して出力するプログラムPになります。nそのものを書くのに、n文字が必要なわけではありません。

nの二進表記がd文字なら、各文字を二度書き、末尾に01を付けることで数の終わりを区切れます。この方法なら、nの記録には2d + 2文字を使います。

探索プログラムの長さ：$|P| \le c + 2d + 2$

探す文字列の長さn	nの二進表記の長さd	nの記録の長さ2d + 2
16	5	12
256	9	20
65,536	17	36

表はnを記録する部分だけを数えています。計算器と探索コードの長さcも、必ず加えます。

④ 十分大きいnでは、二つの条件が衝突します。

nが倍々に増えても、nを二進数で書く桁数dは一つずつしか増えません。cがどれほど大きな固定値でも、十分大きいnでは、c + 2d + 2がnより小さくなります。PもxをつくるプログラムなのでK(x)は、Pの長さを超えられません。

探すときの条件 $K(x) \ge n$ ／ その文字列を作るプログラム $K(x) \le |P| < n$

矛盾から取り下げるのは、「すべての文字列のK(x)を正確に答えて終了する計算器がある」という仮定です。特定の文字列の短いプログラムを探したり、個別の例で最小値を求めたりすることまで不可能なのではありません。

終わることは、終わるのですが。

先生　さっきは、時間がかかっても構わないと言っていたよね？

ラナ　はい。それでも作れない計算器がある、という話でした。今度は、終えられる計算にどれほど時間がかかるかを見てみます。

先生　今度は、待てば終わることは終わるんだよね？

ラナ　はい。ただし、先生が待てる時間のうちに終わるかは、別に考える必要があります。

一文字ずつ、最後まで一度

0 1 0 1 1 0　6文字

0 1 0 1 1 0 0 1 0 1 1 0　12文字

覚える状態

偶数 / 奇数

1を読んだら切り替え

ラナ　最初に、1の個数が偶数かを調べた方法を覚えていますか？ 一文字読むごとに、状態を変えるかを決めました。

先生　偶数か奇数かだけ、覚えておけばよかったはず。

ラナ　はい。文字列が倍の長さになれば、読む文字も倍になります。

先生　コンピュータの速さが分からなくても、やることがどれだけ増えるかは分かるんだ。

ラナ　そのとおりです。入力が長くなるにつれて、必要な計算ステップ数が増える、その増え方を時間計算量ともいいます。

※ 入力の長さは、問題を決められた方式で書いたときの文字数です。同じ長さの入力のうち、最も多くのステップが必要になる場合を考えます。機械の一回の動作を一ステップと数えます。

先生　どんな計算でも、長さが倍なら時間も倍くらいかかるの？

ラナ　そうとは限りません。入力の文字を一つずつ、すべての文字と組にして調べると、組の数は長さの二乗になります。

先生　長さは倍なのに、四倍も調べるの？

ラナ　はい。一方、毎回二つに分かれる選択をすべて調べるなら、選択が一回増えるたびに、場合の数が倍になります。

増え方を比べた例

入力長 n	n	n^2	2^n
10	10	100	1,024
20	20	400	1,048,576
30	30	900	1,073,741,824

先生　長さを少し増やしただけなのに、あちらはもう、こんなに大きな数になるんだ。

ラナ　はい。長さが倍なら四倍、三乗なら八倍になる、といった増え方ですね。このように、入力の長さを何乗かして、一定の数を掛けた回数以内で終えられるとき、多項式時間といいます。

先生　最後のものは、一文字増えるたびに場合の数が倍になっていたし。

ラナ　はい。そうした増え方を、指数的な増加といいます。

先生　それなら、多項式時間で終わる方法があればいいの？

ラナ　はい。多項式時間で「はい」か「いいえ」かを判定できる問題の集まりを、Pといいます。

※ Pはプログラムではなく、問題の集まりです。多項式時間は、すべての入力で成り立つステップ数の上限についての言葉です。必ず実用上速いとは限らず、表の三つがすべての増え方を表すわけでもありません。

各カードは一度ずつ・目標の合計16

3	5	8	11

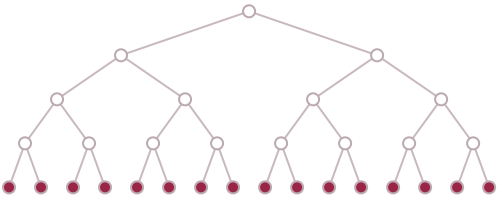
ラナ　先生。ここから何枚か選んで、合計を16にできるでしょうか。各カードは一度ずつしか使えません。

先生　3と8なら11で……あ、5と11を選べばいい。

ラナ　はい。選んだ二枚を足せば、すぐに確認できます。

先生　選ぶときはいろいろ試したのに、合っているかを見るのは足し算一回で終わり？

カードごとに選ぶ／選ばない



四枚 → 16通りの選択

選んだカードを確認

5 + 11 = 16

「ある」という答えの根拠

ラナ　はい。カードが多くなったらどうでしょうか。すべての組み合わせを調べる方法では、一枚増えるたびに場合の数が倍になります。

先生　誰かが選んだカードを見せてくれれば、そのカードだけ足せばいいのに。

ラナ　はい。与えられたカードか、一枚を何度も使っていないかも確認します。数字が長くなれば足し算にも時間がかかりますが、こうした確認は、入力の長さに対する多項式時間でできます。

先生　探すことと、見つけたものを確認することでは、ずいぶん違うんだ。

ラナ　そのとおりです。ここでの問いは「合計を合わせる選び方があるか」です。先生が選んだカードが、「ある」という答えの根拠になります。

※ カードの枚数と、数字を書く長さは違います。入力長には、各カードの数字と目標の合計を書く文字数も含みます。この小さな例だけで、どんな場合も難しいとは結論できません。

先生　逆に、どう選んでも合わない場合は？

ラナ　そのときは、合う組み合わせを示せません。今の確認方法は、「ある」という答えを裏付けるものです。

「ある」という答え＋選んだカードの一覧

与えられたカード？ → 重複使用はない？ → 合計は目標と同じ？

先生　「見つかりませんでした」だけでは、本当にないとは言えないか。

ラナ　はい。このように「はい」という答えに、多項式時間で確認できる根拠がある問題の集まりを、NPといいます。根拠の長さも、入力の長さに対する多項式で制限します。

先生　確認する資料からして、きりがないほど長かったら困るよ。

$P \subseteq NP$

Pの問題はすべてNPにも入る

同じかどうかは未解決。

先生　さっきのPは、答えを直接出せる問題だったよね？ それなら、それもNPに入るの？

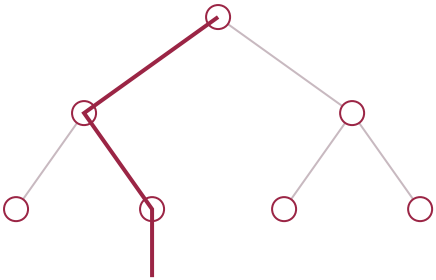
ラナ　はい。直接判定して確認すればよいので、Pの問題はすべてNPにも入ります。

先生　それなら、NPだからといって、全部時間がかかる問題というわけではないんだ。

ラナ　そのとおりです。NPは「多項式時間では解けない」という意味ではありません。Nは、先ほど学んだ非決定性から来ています。

※ NPでは、「はい」となる入力には少なくとも一つの根拠があり、「いいえ」となる入力には確認を通る根拠がないことが必要です。「いいえ」にも同じような短い根拠を要求する定義ではありません。

非決定性モデル・選択と確認



選択の構造の一部を示した概念図

先生　カードごとに、選ぶか選ばないかで分ければ、さっきの計算の木のように描けそう。

ラナ　はい。それぞれの枝で選び終えたら、確認します。正しい選択をした枝が一つでも受理すれば、答えは「はい」です。

ラナ　NPの問題は、このように非決定性チューリングマシンで多項式時間に判定できます。どの枝をたどっても、選んで確認する過程が多項式時間で終わるようにするのです。

先生　それなら、そういう機械でコンピュータを作ればいいんじゃないかな？ 一つずつ探すより速そうだけど。

ラナ　キヴォトスで私たちが使っているコンピュータは、そのようには動きません。非決定性チューリングマシンは、可能な選択を枝に分けて考える、理論上のモデルです。

非決定性モデルの時間

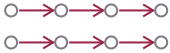
最も長い経路のステップ数

全経路の合計ではありません。

コンピュータで順にたどるなら

①

②

③ 

複数の経路を調べる時間がかかります。

先生　紙に分かれ道を描いても、コンピュータが自動的に全部の道を進んでくれるわけではないんだ。

ラナ　はい。先ほどは、その枝を一つずつたどる方法を見ましたね。一本の枝は短くても、全部たどると時間がかかる場合があります。

先生　それなら、私たちのコンピュータでは、必ず一つずつ試すしかないの？

ラナ　すべての枝を調べずに答えを出す、別の方法があるかもしれません。NPのすべての問題を、決定性チューリングマシンでも多項式時間に判定できるかを問うのが、P対NP問題です。

先生　それは、まだ分からないの？

ラナ　はい。どちらなのかは、まだ分かっていません。

P = NP ?

確認が速ければ、判定も速い？

「速い」の基準は多項式時間

先生　計算できない問題とは違うんだ。できることはできるけれど、どれだけ速くできるかは、まだ分からないということ？

ラナ　はい。ここでは候補が有限個なので、全部確認すればいつか終わります。それより効率のよい方法があるかは、別の問いです。

先生　これで終わりかと思ったら、最後の問題はまだ未解決だなんて。

ラナ　先生の宿題にするつもりではありませんでした。

先生　よかった。それなら、今日はここまで？

ラナ　はい。次は、答えを探すほうも一緒にやってみませんか？

※ ここでは、決定性チューリングマシンで多項式時間に判定するPと、すべての枝が多項式時間で停止する非決定性チューリングマシンで判定するNPを比べました。5章の「計算できる範囲が同じ」だけでは、「同じ時間で計算できる」とはいえません。

ラナ
……思ったより長く話していましたね。
先生
もうこんな時間か。当番も終わりの時間だね。

ラナ
まだ、お見せできていないものもあるのですが……。
先生
今日はありがとう。いろいろ勉強になったよ。

先生
じゃあ、続きはまた今度聞かせてもらおうかな。
ラナ
はい。次の当番のときに、続きをお話ししますね。

参考資料・出典

学習内容

MIT OpenCourseWare
6.045J Automata, Computability, and Complexity (2011)
https://ocw.mit.edu/courses/6-045j-automata-computability-and-complexity-spring-2011/

OpenDSA
Pushdown Automata
https://opendsa-server.cs.vt.edu/ODSA/Books/PIFLA/html/PDA.html

Cornell University · Andrew Myers
CS 4120 Lecture Notes
https://www.cs.cornell.edu/courses/cs4120/2026sp/notes/

Robert Nystrom
Crafting Interpreters: Scanning / Parsing Expressions / Evaluating Expressions
https://craftinginterpreters.com/contents.html

York College of Pennsylvania
CS 340, Lecture 7: Turing Machines (2016)
https://ycpcs.github.io/cs340-fall2016/lectures/lecture07.html

Duke University
COMPSCI 334: Section — Turing Machines
https://courses.cs.duke.edu/cps140/compsci334/spring23/lects/sectntmS.pdf

Pramod Ganapathi · Stony Brook University
Theory of Computation (Turing Machines)
https://www3.cs.stonybrook.edu/~pramod.ganapathi/doc/theory-of-computation/TuringMachines.pdf

MIT 6.045
Lecture 9: Turing Machines (2020)
https://people.csail.mit.edu/rrw/6.045-2020/lec9-before-class.pdf

Nicolas Resch · Carnegie Mellon University
CS 252, Lecture 4: Kolmogorov Complexity (2020)
https://www.cs.cmu.edu/~venkatg/teaching/15252-sp21/notes/Kolmogorov-Complexity.pdf

Clay Mathematics Institute
P vs NP
https://www.claymath.org/millennium/p-vs-np/

原作・キャラクター

ブルーアーカイブ -Blue Archive-
キャラクター・世界観、公式立ち絵、ゲーム内の先生の絵
https://bluearchive.jp/

Blue Archive Utils · キャラクター紹介・ボイス台詞
https://blue-utils.me/student/rana?lang=ja https://blue-utils.me/tool/voice

ブルーアーカイブ攻略有志Wiki · ラナ
https://bluearchive.wikiru.jp/?ラナ